

# Module 08: Records and Enums

---

Module 06 made you write `equals`, `hashCode` and `toString` by hand, and showed a `HashSet` losing its contents when you got it wrong. This module is the answer to that problem.

Records carry data. Enums fix a set of instances. Between them they replace a large amount of the hand-written boilerplate that older Java is famous for.

## What You'll Learn

---

- What a record generates, shown with `javap` rather than `claimed`
- Compact constructors, for validating and normalising
- Why "immutable" for a record is shallower than it sounds
- Enums as classes with fixed instances, carrying state and behaviour
- Why `ordinal()` is a trap
- How to choose between a class, a record and an enum

## Coming From Another Language

---

|                        | Python                                             | Java                                                   |
|------------------------|----------------------------------------------------|--------------------------------------------------------|
| Data holder            | <code>@dataclass</code> or <code>NamedTuple</code> | <code>record</code>                                    |
| Equality               | generated by the decorator                         | generated by the record                                |
| Fixed instance set     | <code>enum.Enum</code>                             | <code>enum</code> , but each constant is a full object |
| Per-constant behaviour | awkward                                            | an abstract method each constant implements            |
| Immutability           | <code>frozen=True</code> , still shallow           | records, also still shallow                            |

The shallowness caveat applies in both languages, and for the same reason. A frozen container holding a mutable list still holds a mutable list.

## The Lesson

---

### What a record is

```
record Point(int x, int y) { }
```

That one line gives you a constructor, `equals`, `hashCode`, `toString`, and an accessor per component. From [Records.java](#):

```
toString: Point[x=3, y=4]
equals:    true
hashCode: true
set size: 1
```

The last line is the point. Module 06's hand-written `BadPoint` put two equal objects in a `HashSet` and then could not find either. A record cannot make that mistake.

Use the Module 01 technique to see what was generated. For `public record Money(long cents, String currency) { }`:

```
public final class Money extends java.lang.Record {
    public Money(long, java.lang.String);
    public final java.lang.String toString();
    public final int hashCode();
    public final boolean equals(java.lang.Object);
    public long cents();
    public java.lang.String currency();
}
```

Three things follow from that:

- **final class** — records cannot be extended, ever.
- **extends Record** — every record already has a superclass, so it cannot extend anything else. It can implement interfaces freely.
- **Accessors have no get prefix.** It is `p.x()`, not `p.getX()`.

A record may have extra methods, static factories and interfaces. What it may **not** have is extra instance state:

```
record Bad(int x) { private int cached; } // does not compile
```

That restriction is the guarantee. A record *is* its components, so two records with equal components are interchangeable.

## Compact constructors

Where a record validates and normalises. From `CompactConstructors.java`:

```
record Email(String address) {
    Email {
        if (address == null || !address.contains("@")) {
            throw new IllegalArgumentException("not an email address: " + address);
        }
        address = address.strip().toLowerCase();
    }
}
```

No parameter list, no field assignment. The compiler assigns the parameters to the fields **after** this body runs, so reassigning the parameter changes what gets stored. Writing `this.address = ...` here is a compile error.

Normalisation matters more than it looks. A record's `equals` compares components, so if two logically equal values can have different components, the record is lying:

```
new Fraction(6, 8)    -> 3/4
new Fraction(-1, -2) -> 1/2
```

Without reducing and sign-normalising in the constructor, `6/8` and `3/4` would be unequal despite being the same number. **When a record represents a concept rather than a raw tuple, normalise in the compact constructor.**

## Shallow immutability

A record's fields are `final`. That is all the language promises. From [ShallowImmutability.java](#):

```
--- the leak ---
built with: Leaky[title=post, tags=[draft]]
after the caller edited their own list: Leaky[title=post, tags=[draft, smuggled-in]]
after editing the accessor's result:    Leaky[title=post, tags=[draft, smuggled-in,
smuggled-out]]

--- sealed off ---
built with: Safe[title=post, tags=[draft]]
after the caller edited their own list: Safe[title=post, tags=[draft]]
the accessor's list refused modification
```

Two separate leaks, closed by one line:

```
record Safe(String title, List<String> tags) {
    Safe {
        tags = List.copyOf(tags);
    }
}
```

`List.copyOf` copies, so the caller's list is no longer yours, and returns an immutable list, so the accessor is safe to hand out directly.

This matters more for records than for ordinary classes, because records are used as map keys and set elements precisely because their `equals` is free. Module 06's mutation trap applies in full: put a `Leaky` in a `HashSet`, edit its list, and it is lost inside the set that still counts it.

**Write `List.copyOf` in the compact constructor every time a component is a collection.**

## Enums

An enum is not a list of integer constants. Each constant is a full object with its own state and, if you want, its own behaviour. From `Enums.java`:

```
MERCURY  gravity  3.70  a 75kg person weighs  277.7 N
EARTH    gravity  9.80  a 75kg person weighs  735.2 N
MARS     gravity  3.71  a 75kg person weighs  278.4 N
```

**Constant-specific behaviour** is the feature worth knowing:

```
enum Operation {
    PLUS("+") { @Override double apply(double a, double b) { return a + b; } },
    MINUS("-") { @Override double apply(double a, double b) { return a - b; } };

    abstract double apply(double a, double b);
}
```

Adding a constant without its `apply` is a compile error. The alternative, a `switch` somewhere else, has to be found and updated by hand.

**`ordinal()` is a trap.** It is the declaration position, so inserting a constant silently renumbers everything after it:

```
PENDING -> stored as P, ordinal happens to be 0
ACTIVE   -> stored as A, ordinal happens to be 1
FAILED   -> stored as X, ordinal happens to be 2
```

Never persist an ordinal and never switch on one. Give the constant an explicit code field, as `Status` does.

Enums are the one place where `==` is idiomatic, because each constant is a singleton.

## Choosing the shape

| Use                 | When                                                                          |
|---------------------|-------------------------------------------------------------------------------|
| <code>record</code> | the type is its data, and two instances with the same data are the same thing |
| <code>enum</code>   | the set of instances is fixed and known at compile time                       |
| <code>class</code>  | it has identity, mutable state, or behaviour that outweighs its data          |

The question that settles it: **if two instances hold identical values, are they interchangeable?** Two `Point(3, 4)` are. Two bank accounts with the same balance are not.

## Run It

---

```
java modules/08-records-and-enums/examples/Records.java
java modules/08-records-and-enums/examples/CompactConstructors.java
java modules/08-records-and-enums/examples/ShallowImmutability.java
java modules/08-records-and-enums/examples/Enums.java

./scripts/verify-examples.sh modules/08-records-and-enums
```

## Common Mistakes

---

**Putting a mutable collection in a record and calling it immutable.** Two leaks, one line to fix.

**Persisting `ordinal()`.** Someone inserts a constant in the middle and every stored row now means something else.

**Using a record for something with identity.** A `User` record means two users with the same name and email are the same user. Sometimes true, usually not.

**Forgetting to normalise.** A record whose `equals` compares raw components will treat `6/8` and `3/4` as different fractions.

**Writing `this.x = x` in a compact constructor.** It does not compile. Assign to the parameter.

**Writing a `switch` over an enum in five places instead of a method on the enum.** Constant-specific behaviour keeps the logic with the data and turns an omission into a compile error.

## Key Takeaways

---

- **A record is its components.** The compiler writes `equals`, `hashCode`, `toString` and accessors, correctly.
- **Records are final and already extend `Record`,** so no inheritance, but interfaces are fine.
- **Compact constructors validate and normalise,** and assign to the parameter.
- **Immutability is shallow.** `List.copyOf` a collection component every time.
- **Enum constants are objects** and can carry state and per-constant behaviour.
- **Never use `ordinal()` for anything durable.**
- **Pick a record when equal data means equal thing,** an enum when the instances are fixed, a class otherwise.

## Homework

---

[homework/README.md](#)

Model a small order system with records and enums, then close two immutability leaks you created on purpose. Reference solution in [solutions/08-records-and-enums/](#).